



Disciplina de Automação de Software e Infraestrutura
Prof. Dr. Eden Ricardo Dosciatti
edenrd@utfpr.edu.br

PROJETO INTEGRADOR FINAL:

Engenharia DevOps & Observabilidade

1. O Cenário (*Briefing*)

A sua equipe acaba de assumir o controle do cargo de **Engenheiro DevOps Júnior** na startup **TechVotos**.

A empresa possui uma micro-aplicação crítica: uma **API de Votação** (ex: votar entre "**Opção A**" e "**Opção B**"). O objetivo da sua equipe é garantir que essa aplicação seja **versionada**, **containerizada**, tenha um **pipeline de CI básico** e, o mais importante, seja **monitorada em tempo real** para que a equipe saiba quantos votos estão entrando e se o sistema está **online**.

A equipe de Desenvolvimento (**Dev**) finalizou a criação da API, escrita em **Python/Flask**. O código funciona na máquina deles, mas eles não possuem conhecimento sobre infraestrutura, containers ou monitoramento.

A Missão:

O Líder Técnico (Professor) lhe entregou os arquivos da aplicação e definiu o seguinte objetivo:

"Precisamos profissionalizar essa entrega. O código deve ser versionado corretamente, rodar isolado em containers Docker e precisamos de um Painel de Controle (Dashboard) para a diretoria acompanhar a votação e a saúde do sistema em tempo real."

2. Recursos Fornecidos (Kit do Desenvolvedor)

Abaixo estão os arquivos que o time de **Dev** lhe entregou. **A tarefa da equipe é criar a infraestrutura ao redor deste código.** Você não precisa (e não deve) alterar a lógica de programação do arquivo **app.py**.

Utilizando o ambiente Linux (Debian 12) e o **VSCode**, crie, no terminal, um **diretório** para este projeto, com o **nome da equipe**, e crie os seguintes arquivos dentro dele:

Arquivo: app.py → A aplicação web (API). Programa em Python

```
from flask import Flask, Response
from prometheus_client import Counter, generate_latest

app = Flask(__name__)

# Métrica para o Prometheus monitorar
VOTOS = Counter('app_votos_total', 'Total de votos computados',
['opcao'])

@app.route('/')
def home():
    return "<h1>Sistema de Votação</h1><p>Vote acessando: <a href='/votar/a'>/votar/a</a> ou <a href='/votar/b'>/votar/b</a></p>"

@app.route('/votar/<opcao>')
def votar(opcao):
    if opcao.lower() in ['a', 'b']:
        VOTOS.labels(opcao=opcao.lower()).inc()
        return f"Voto computado para: {opcao.upper()}!"
    return "Opção inválida! Tente A ou B.", 400

@app.route('/metrics')
def metrics():
    # Rota que expõe as métricas para o Prometheus
    return Response(generate_latest(), mimetype='text/plain')

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

Arquivo: requirements.txt → As dependências necessárias para a aplicação rodar.

```
flask
prometheus_client
```

3. Requisitos Técnicos (O que deve ser feito)

Com esses dois arquivos em mãos (ao final deste documento uma imagem deles no VSCode, para qualquer dúvida), a responsabilidade da equipe é puramente de **Infraestrutura e Automação**. A equipe não precisa programar, somente **empacotar**.

No mesmo **diretório** criando no item 2, execute as etapas abaixo:

Etapa 1: Ambiente Linux (Debian 12)

1. Todo o desenvolvimento e execução devem ser feitos no ambiente Linux configurado durante o semestre.

Etapa 2: Controle de Versão (Git/GitHub)

1. Inicie um repositório **Git** local no diretório do projeto.
2. **Obrigatório:** Crie um arquivo **.gitignore** na raiz do projeto para garantir que arquivos temporários (como o diretório **_pycache_** ou ambientes virtuais **venv**) **não** sejam enviados ao repositório. Isso é comum na linguagem Python.
3. Realize **commits** organizados (ex: "**Configuração do Docker**", "**Adição do Pipeline**", ...).
4. Ao final, envie todo projeto para o GitHub.
5. Qual o endereço do GitHub? _____

Etapa 3: Containerização (Docker)

1. Crie um **Dockerfile** otimizado para empacotar a aplicação:
 - Utilize uma imagem base leve (sugestão: **python:3.9-slim**).
 - Instale as dependências listadas no **requirements.txt**.
 - Exponha a porta correta (consulte o **app.py** para verificar a porta).
 - Defina o comando para iniciar a aplicação.

Etapa 4: Orquestração (Docker Compose)

Crie um arquivo **docker-compose.yml** que suba o ambiente completo com 3 serviços na mesma rede:

1. **app-votacao**: A aplicação construída a partir do seu **Dockerfile**.
2. **prometheus**: O coletor de métricas.
 - Crie e configure o arquivo **prometheus.yml** para ler os dados do serviço **app-votacao** a cada **5 segundos**.
3. **grafana**: A interface de visualização.

Etapa 5: Pipeline de Automação (GitLab CI)

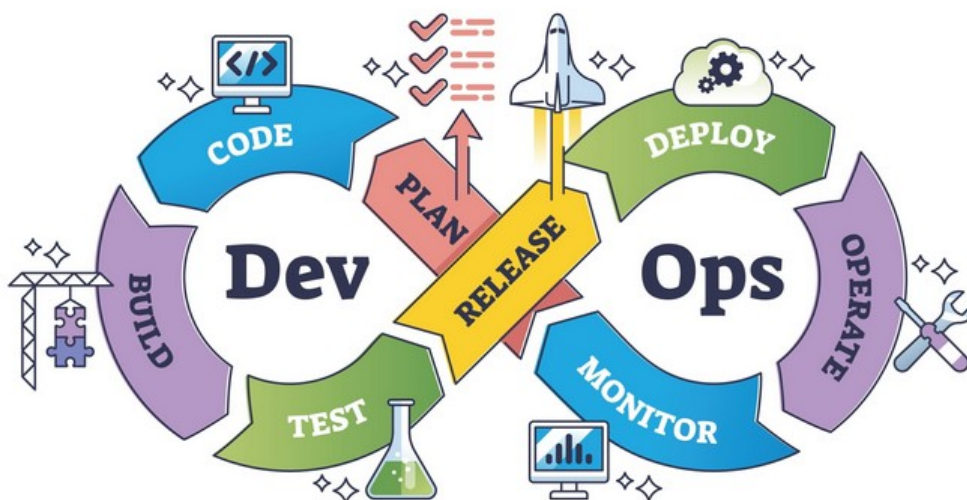
1. Crie um arquivo **.gitlab-ci.yml**.
 - O pipeline deve conter um estágio de **Test** ou **Build** que verifique se a sintaxe do Python está correta **OU** se o container é construído com sucesso.

Etapa 6: Monitoramento (Prometheus)

1. A aplicação deve expor uma rota **/metrics** (formato Prometheus).
2. Configurar o **Prometheus** para ler esses dados.

Até estas etapas, o diretório de trabalho deve conter estes arquivos:

```
drwxr-xr-x 3 devops devops 4096 dez  2 16:14 .
drwxr-xr-x 3 devops devops 4096 dez  2 16:01 ..
-rw-r--r-- 1 devops devops  728 dez  2 16:01 app.py
-rw-r--r-- 1 devops devops  493 dez  2 16:01 docker-compose.yml
-rw-r--r-- 1 devops devops  164 dez  2 16:01 Dockerfile
drwxr-xr-x 8 devops devops 4096 dez  2 16:15 .git
-rw-r--r-- 1 devops devops   29 dez  2 16:14 .gitignore
-rw-r--r-- 1 devops devops  199 dez  2 16:01 .gitlab-ci.yml
-rw-r--r-- 1 devops devops  133 dez  2 16:01 prometheus.yml
-rw-r--r-- 1 devops devops   24 dez  2 16:01 requirements.txt
```



Aqui está o checklist claro do que cabe a equipe fazer:

1. **Ler o código:** Identificar qual porta a aplicação usa (**5000**).
2. **Dockerizar:** Criar o **Dockerfile** que copia esses arquivos e instala o **requirements.txt**.
3. **Orquestrar:** Criar o **docker-compose.yml** para subir a aplicação junto com o **Prometheus** e **Grafana**.
4. **Automatizar:** Criar o **.gitlab-ci.yml** para testar se o container sobe.
5. **Monitorar:** Configurar o painel no Grafana para ler a métrica **app_votos_total**.

Após todos os arquivos de configuração montados, faça o passo a passo para verificar se está rodando seu projeto:

1. **Execução:** No terminal, dentro do diretório do projeto integrados, rode o **docker compose up -d --build**.
2. **Gerar Tráfego:** Acesse o navegador:
 - Em uma aba: **http://localhost:5000/votar/a** → Aqui, faça reload várias vezes para ir somando os votos.
 - Em outra aba: **http://localhost:5000/votar/b** → Aqui, faça reload várias vezes para ir somando os votos.
3. **Prometheus:** Em outra aba: **http://localhost:9090/targets**
 - Verifique o status do endpoint **app-votacao:5000** deve estar **UP** (Verde). Se estiver vermelho, o Prometheus não achou o container da app.

Arquivos do VSCode:

```
app.py > ...
1  from flask import Flask, Response
2  from prometheus_client import Counter, generate_latest
3
4  app = Flask(__name__)
5
6  VOTOS = Counter('app_votos_total', 'Total de votos computados', ['opcao'])
7
8  @app.route('/')
9  def home():
10     return "<h1>API Votacao Online</h1><p>Rotas: <a href='/votar/a'>/votar/a</a> ou <a href='/votar/b'>/votar/b</a></p>"
11
12 @app.route('/votar/<opcao>')
13 def votar(opcao):
14     if opcao.lower() in ['a', 'b']:
15         VOTOS.labels(opcao=opcao.lower()).inc()
16         return f"Voto registrado para: {opcao.upper()}"
17     return "Opcao invalida!", 400
18
19 @app.route('/metrics')
20 def metrics():
21     return Response(generate_latest(), mimetype='text/plain')
22
23 if __name__ == '__main__':
24     app.run(host='0.0.0.0', port=5000)
```

requirements.txt

```
1  flask
2  prometheus_client
```